Grounding the AI Scrum Master: A Sprint-Replay Test and Pilot

Hani AbuSharkh
Higher Colleges of Technology
Al Ain, UAE
habusharkh@hct.ac.ae

***Abstract*—Agent systems that carry the Scrum Master label are multiplying, yet none we found has been judged against what actually happened in real sprints, while work that predicts sprint outcomes from Jira histories stops at prediction and never acts. We make four contributions. First, we map each Scrum Master accountability to the best published evidence and show that the label has run ahead of it. Second, we name two failure modes that bound the design space: an ungrounded agent says things the data does not support, and an over-grounded one repeats its model and adds nothing. Third, we propose sprint replay, a hindsight-free test on public data: rebuild the board as it stood, run a grounded agent against a model-only arm and an agent-only arm, and report an agreement rate and a cost-weighted measure of what grounding adds. Fourth, we run a pilot on 40 held-out TAWOS sprints at two checkpoints. A Claude agent given the risk model agreed with it on 91% of decisions and cost the same at every price of a warning, while the same agent without the model warned less and agreed on 73%. One warning in five advised closing the sprint on time with work left over, which meets the label without delivering more. The test is cheap, and it already catches the over-grounded failure.**

***Index Terms*—LLM agents, Scrum, agile software development, sprint analytics, evaluation, grounding**

I. Introduction

Much of what a Scrum Master does never reaches the tracker. Shastri *et al.* [1] name six everyday functions of the role: facilitating, mentoring, negotiating, adapting processes, coordinating and protecting the team. Only two leave much of a trace in Jira. The rest happen in conversation.

Language-model agents have nonetheless started to wear the title. Since 2024, published systems have given an agent the Scrum Master role in a code-generation pipeline [2], simulated agile teams with role agents [3], and turned burndown charts into status reports under the name of an AI Scrum Master [4].

Our position is that the label has run ahead of the evidence, and that a specific, affordable test can bring it back into line. The test is sprint replay: rebuild a project's board as it stood at a chosen moment, give the agent only what was knowable then, let it decide whether and what to say, and score that against what the sprint went on to do. No partner organisation or live team is needed. What is needed is a sensing layer the agent can cite, and the discipline to report how often the agent merely repeats it.

This paper makes four contributions:

- a map of the role's accountabilities against the best published evidence, which shows the label running ahead of it (Section II);
- two documented failure modes, ungrounded and over-grounded, that bound what a credible system must avoid (Section III);
- sprint replay, a hindsight-free test with a model-only arm, an agent-only arm, an agreement rate and a cost-weighted measure of added value (Sections IV and V);
- a pilot of that test on 40 public sprints, which already shows one of the two failure modes (Section VI).

II. The Label and the Evidence

Systems that use Scrum roles fall into three families. The first organises code generation: AgileCoder's Scrum Master agent reviews a backlog, picks sprint tasks and judges deliverability, and is scored on code benchmarks alone [2]. The role supplies a vocabulary, not a set of responsibilities. The second simulates agile teams; CogniSim models scaled-agile roles with language-model agents but has no Scrum Master agent at all, and its own future work asks for predictive analytics for delays [3]. The third targets the role directly, and here the evidence is real but small. An action-research study put assistants into daily Scrums and refinement for three teams at a large organisation: about a third of early refinement outputs were wrong because the input data was stale, and participants asked that it inform rather than warn [5]. Another study prompts models to write status reports from burndown charts and scores the text with similarity metrics, with no agent and no prediction [4].

Software management accounts for under one percent of language-model usages in a review of 395 software-engineering papers [6], and a review of Scrum-specific work finds no applications in the implementation or review-and-retrospect phases [7]. Table I sets the role's accountabilities against the best evidence we found.

Three patterns follow. The more relational the task, the thinner the evidence: of the six functions Shastri *et al.* [1] identify, only facilitation and coordination have any. Evaluation habits are weak, with small samples, single runs and author-labelled ground truth, and where run-to-run stability has been measured it is poor [11]. And the data is lopsided. Public sprint histories are plentiful, with one dataset holding thousands of sprints and millions of change-log entries [12], while public transcripts of stand-ups, reviews and retrospectives are, as far

TABLE I
SCRUM MASTER ACCOUNTABILITIES AGAINST THE BEST EVIDENCE WE FOUND. ROWS WITHOUT A CITATION REFLECT ABSENCES REPORTED BY THE REVIEWS DISCUSSED IN THE TEXT.

Accountability	Coverage	Best evidence
Effort and story-point estimation	Well covered	Real data, many projects [8]
Delivery-risk prediction at planning	Moderate	Real Jira histories [9], [10]
In-sprint monitoring	Thin	Checkpoint prediction, no agent [9]
Daily Scrum facilitation	Thin	One field site, three teams [5]
Impediment detection and removal	Thin to empty	Detection only; no removal studies
Review and stakeholder reporting	Near empty	Text-similarity scoring [4]
Retrospectives, coaching, team health	Near empty	Small author-labelled studies
End-to-end agent in the role	Label only	Code benchmarks and demos [2]

as we can tell, absent. The half of the role a tracker records is therefore the half in which an agent can be held to account today.

III. TWO WAYS TO FAIL

a) Ungrounded: Asked certification-style Scrum questions, current models recall well and interpret poorly; in one evaluation at least one prompt failed on roughly a sixth of the items, with errors clustering on interpretive topics [13]. Generated sprint plans were judged unfit for direct use, with priority accuracy swinging from 44 to 92 percent across runs [11]. In mission-critical risk analysis, retrieval-grounded models had the lowest hallucination rates [14]. In this role the characteristic failure is a confident wrong statement, and the characteristic fix is grounding.

b) Over-grounded: The opposite failure is less discussed and, for research, more dangerous. Given a predictive model as a tool, agents tend to copy it: in one study they reproduced a graph model's output on 97.6 to 99.2 percent of cases, and stronger backbones deferred more, not less [15]. An agent that repeats its model is the model with added latency. Any claim of an agent in this role should therefore report its agreement with the raw model and include a model-only arm.

Neither failure is particular to Scrum. Agents are being placed over classical predictive models in clinical triage, credit decisions and operations, and the same two questions arise each time: when should the agent believe the model, and when is acting on it worth the cost? Sprints are an unusually good place to study both, because outcomes are public, time-stamped, and free of hindsight once the board is rebuilt.

IV. WHAT GROUNDING MEANS HERE

Grounding means that the agent's statements about risk trace back to models fitted on real sprint histories, under an evaluation that does not leak the future. Some sprint-outcome studies report accuracies near or above 0.9 using features visible only once the sprint is under way [16], or after resampling with no stated split order [17], and leakage taxonomies now list software engineering among the affected fields [18]. A stricter pipeline produces lower numbers, and the lower numbers are the trustworthy ones. In one study on public Jira data, shuffling sprints before splitting raised late-closure AUC from 0.70 to 0.77. An agent that cites an inflated model inherits the inflation and adds a confident voice to it.

Grounding also limits what the agent may suggest. In defect prediction, guidance derived from explanation methods often fails to fix what it predicts and differs from what developers actually change [19], while restricting suggestions to changes with historical precedent brings them much closer to real practice [20]. Carried into sprints, an agent should not propose cutting scope merely because the model's expectation falls; it should propose changes that comparable teams actually made.

What is left for the agent is narrower than the label and more interesting than the model: reading the issue text the tabular model never sees, deciding whether a situation is worth raising, choosing the moment, and wording it so that a team can act without feeling audited. Each of these is measurable.

V. SPRINT REPLAY AS THE BAR

a) Construction: Public datasets record field-level changes with timestamps [12], which is enough to rebuild a board as it stood at any point in a sprint. For sprint s and checkpoint $c \in \{0, 0.25, 0.5, 0.75\}$ of the planned duration, let $B_{s,c}$ be the board rebuilt from events time-stamped no later than that point. Every arm sees $B_{s,c}$ and nothing later.

b) Arms: Three at minimum: the risk model alone (M), the language-model agent without the model (L), and the grounded agent (G). Without M, no one can tell whether the agent added anything; without L, no one can tell whether grounding did. At each replay point, arm a emits a decision $d_a(s,c) \in \{\textit{warn}, \textit{silent}\}$ and, for L and G, the message it would send.

c) Agreement: Over the set P of replay points,

$$A = \frac{1}{|P|} \sum_{(s,c)\in P} \mathbf{1}\big[d_G(s,c) = d_M(s,c)\big],$$

reported as a first-class result, not a footnote.

d) Added value: We borrow the cost structure of alarm-based prescriptive process monitoring [21]. A warning costs κ, a failed sprint costs F, and a warning raised at checkpoint c averts a fraction $e(c)$ of that cost, with e falling as the sprint proceeds. Let $y_s = 1$ if sprint s failed, $w_a(s) = 1$ if arm a warned at any checkpoint, and $c^*_a(s)$ the first checkpoint at which it did. Then

$$C_a(s) = \kappa\, w_a(s) + F\, y_s\big(1 - e(c^*_a(s))\, w_a(s)\big),$$
$$\Delta = \min\big(\bar{C}_M, \bar{C}_L\big) - \bar{C}_G,$$

where $\bar{C}_a$ is the mean of $C_a(s)$ over sprints. A positive Δ means the grounded agent beats the better of the two baselines. Because κ/F and e differ between teams, Δ is reported as a curve over the cost ratio rather than as one number. Read together, a high A with $\Delta \approx 0$ says the agent is the model, while $\Delta > 0$ with A well below one is evidence of judgement.

TABLE II
PILOT REPLAY ON 40 HELD-OUT TAWOS SPRINTS (20 LATE), PER ARM AND CHECKPOINT. A: SHARE OF DECISIONS EQUAL TO THE MODEL'S.

c	Arm	Warns	Sens.	Spec.	F1	AUC	A
0	M model	.58	.75	.60	.70	.72	1
0	L agent	.40	.55	.75	.61	.69	.73
0	G grounded	.60	.75	.55	.68	.74	.93
0.5	M model	.65	.75	.45	.65	.71	1
0.5	L agent	.43	.60	.75	.65	.70	.73
0.5	G grounded	.60	.75	.55	.68	.72	.90

e) Reliability and calibration: Language-model arms are rerun and their variance reported, and any probability an arm states is checked for calibration. So are alarm thresholds: in that study, one threshold caught 11% to 86% of late sprints on ten new projects.

f) What replay cannot do: Replay measures what an agent would have said, not whether saying it would have changed the sprint. The one controlled study of language-model facilitation found it drew out more information, mostly from quieter participants, but not better decisions [22]. Replay is a filter, not a verdict: a system that fails it should not reach a team, and one that passes has earned a study, not a deployment.

VI. A PILOT REPLAY

To show that the test is cheap, we ran it at small scale on late closure: whether Jira closes a sprint more than a day after its planned end.

a) Setup: We drew 40 sprints from the held-out part of TAWOS [12], the last 30% of each project's history: 20 that closed late and 20 that did not, among the 472 held-out sprints that had committed work and were still open at mid-sprint (seed 42). Each was replayed at planning ($c = 0$) and halfway ($c = 0.5$), so every arm made 80 decisions. Arm M is gradient boosting from our earlier leakage-aware pipeline on the same data, trained only on the first 70% of each project: planning features at $c = 0$ (held-out AUC 0.751 on all 786 test sprints) and planning plus progress features at $c = 0.5$ (0.757 on 775). It warns when its score passes the threshold that maximises sensitivity plus specificity on out-of-fold training scores. Arms L and G are Claude (model claude-opus-5-5), each given the rebuilt board as text: team history, the committed issues with their type and story points and, at $c = 0.5$, which issues were resolved, moved out or added. Project names, issue keys and calendar dates are hidden. G also sees M's probability, threshold and call, and is told it may overrule the model. Boards went out in batches of 20 in random order, one batch per arm and checkpoint. The agents had no tools beyond reading their prompt, and the run logs show one file read and no other call. Each answer gives a warn or silent decision, a probability, and the message the agent would send. Costs use $e(0) = 0.5$ and $e(0.5) = 0.25$ and are reweighted to the 28% late rate of the 472 eligible sprints. Intervals come from 2,000 bootstrap resamples of the 40 sprints.

b) Agreement and added value: Table II gives the result. The grounded agent agreed with the model on 91% of replay points (95% CI 84–98%). It overruled the model 7 times in 80 and was right in 4 of them. The agent without the model agreed with it on 73% (60–85%) and warned less often. On ranking, no arm separates from another: every AUC difference from M is within 0.03, with intervals reaching about ± 0.15. On cost, G and M are level at every price of a warning from $\kappa/F = 0.02$ to 0.5; the gap never exceeds $0.02F$ per sprint, and every interval contains zero. The more cautious L arm gets cheaper as warnings get dearer: from $\kappa/F = 0.2$ upward Δ is negative, and clearly so at 0.5, where it is $-0.086F$ $[-0.161, -0.019]$. In the terms of Section V, this is a high A with $\Delta \leq 0$. On these boards, the grounded agent is the model.

c) What the agents advised: The agent arms sent 81 warnings. We read all of them. Most asked the team to trim scope or re-plan, citing the board's own numbers. But 16 (8 in each arm) advised closing the sprint on its planned date and carrying unfinished work over. That would meet the late-closure label by changing bookkeeping, not delivery. It is the kind of suggestion Section IV warns about, and a replay that scored decisions alone would have missed it. Grounding has to cover what an agent recommends, not only what it predicts.

d) Limits of the pilot: This is one run of one model on 40 sprints, with a balanced sample and wide intervals. The TAWOS extract we used has no issue titles or descriptions, so the agents could not use the text that a tabular model never sees, which is where we expect an agent to add most. Issue type and story points are the values recorded in the dataset, and $e(c)$ is assumed. The pilot shows that the test runs, cheaply, and that it can catch an agent repeating its model. It does not show that agents cannot add value.

VII. LIMITS

Public sprint data is open-source, and enterprise teams may behave differently. Models may have seen public tracker data in pre-training, which is one more reason for the L arm and for replication outside software. The relational half of the role stays out of reach while no public ceremony corpus exists. Finally, autonomy: developers keep planning and management work at low autonomy levels [23], and the one field study in this role reports participants asking to be informed rather than warned [5]. An agent built to inform is easier to accept, and to evaluate honestly, than one built to decide.

VIII. CONCLUSION

The AI Scrum Master exists as a label and a few pilots, not yet as a system whose judgement has been measured against real sprints, while sprint-prediction work senses without acting. The join can be tested cheaply: replay real sprints, ground the agent in models whose evaluation does not leak the future, and report both how often the agent repeated its model and what it added when it did not. Our pilot did exactly that for 80 decisions per arm. The grounded agent agreed with its model nine times in ten and cost the same. The next step is the same test with issue text, more runs and more sprints, which is where an agent should earn its place.

REFERENCES

[1] Y. Shastri, R. Hoda, and R. Amor, "Spearheading agile: The role of the scrum master in agile projects," *Empir. Softw. Eng.*, vol. 26, no. 1, p. 3, 2021.

[2] M. H. Nguyen, T. P. Chau, P. X. Nguyen, and N. D. Q. Bui, "AgileCoder: Dynamic collaborative agents for software development based on agile methodology," in *Proc. IEEE/ACM Int. Conf. AI Foundation Models and Software Engineering (FORGE)*, 2025.

[3] K. Cinkusz and J. A. Chudziak, "Agile software management with cognitive multi-agent systems," in *Proc. 17th Int. Conf. Agents and Artificial Intelligence (ICAART)*, 2025.

[4] G. Alliata, A. Singhal, and R. Bozagiu, "The AI scrum master," in *Agile Processes in Software Engineering and Extreme Programming: Workshops (XP 2024)*, LNBIP, vol. 524. Springer, 2024.

[5] B. Cabrero-Daniel *et al.*, "Exploring human-AI collaboration in agile: Customised LLM meeting assistants," in *Agile Processes in Software Engineering and Extreme Programming (XP 2024)*, LNBIP. Springer, 2024.

[6] X. Hou *et al.*, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, 2024.

[7] D. Peras *et al.*, "Artificial intelligence in Scrum: A systematic literature review," *Informatica*, 2026.

[8] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for estimating story points," *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 637–656, 2019.

[9] M. Choetkiertikul, H. K. Dam, T. Tran, A. Ghose, and J. Grundy, "Predicting delivery capability in iterative software development," *IEEE Trans. Softw. Eng.*, vol. 44, no. 6, pp. 551–573, 2018.

[10] M. Choetkiertikul, P. Banyongrakkul, C. Ragkhitwetsagul, S. Tuarob, H. K. Dam, and T. Sunetnanta, "Sprint2Vec: A deep characterization of sprints in iterative software development," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 220–242, 2025.

[11] Y. Yoon, K. Iwan, M. Zwart, X. Qin, H. Lee, and M. Spichkova, "Scrum sprint planning: LLM-based and algorithmic solutions," arXiv:2512.18966, 2025.

[12] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, "A versatile dataset of agile open source software projects," in *Proc. 19th Int. Conf. Mining Software Repositories (MSR)*, 2022, pp. 707–711.

[13] M. Perkusich *et al.*, "Evaluating large language models on professional scrum master assessments," arXiv:2607.00049, 2026.

[14] M. Esposito, F. Palagiano, V. Lenarduzzi, and D. Taibi, "Beyond words: On large language models actionability in mission-critical risk analysis," in *Proc. 18th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2024.

[15] Z. Wang and P. Vemuri, "When the tool decides: LLM agents defer blindly to graph neural network tools, and stronger backbones defer more," arXiv:2606.14476, 2026.

[16] Y. J. Pérez Castillo, S. D. Orantes Jiménez, and P. O. Letelier Torres, "Sprint management in Agile approach: Progress and velocity evaluation applying machine learning," *Information*, vol. 15, no. 11, p. 726, 2024.

[17] P. G. Obike, V. E. Ekong, and O. U. Obot, "A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning," *Int. J. Comput. Appl.*, vol. 187, no. 49, pp. 40–53, 2025.

[18] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[19] G. Lee and S. U.-J. Lee, "Can we trust the actionable guidance from explainable AI techniques in defect prediction?" in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2025.

[20] K. Oueslati, G. Laberge, M. Lamothe, and F. Khomh, "Mining action rules for defect reduction planning," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[21] S. A. Fahrenkrog-Petersen, N. Tax, I. Teinemaa, M. Dumas, M. de Leoni, F. M. Maggi, and M. Weidlich, "Fire now, fire later: Alarm-based systems for prescriptive process monitoring," *Knowl. Inf. Syst.*, vol. 64, no. 2, pp. 559–587, 2022.

[22] M. Alsobay, D. M. Rothschild, J. M. Hofman, and D. G. Goldstein, "Bringing everyone to the table: An experimental study of LLM-facilitated group decision making," arXiv:2508.08242, 2025.

[23] R. Choudhuri, C. Bird, C. Badea, M. Gerosa, and A. Sarma, "You shall not pass! Where and why developers draw the line on AI autonomy," arXiv:2607.00533, 2026.